ADS MANAGEMENT SYSTEM

Master Developer Handoff Document

Document type	Technical handoff — developer execution
Scope	Part 3: Analytics & Monitoring | Part 4: Conversion Tracking
Brands	All brands — build for primary brand first, replicate for all others
Version	1.0
Status	Complete — ready for development

IMPORTANT — READ BEFORE DEVELOPMENT STARTS

Build in the exact sequence defined in this document. Do not skip sections or build out of order. Each sprint builds on the one before it. All platforms must be upgraded to their required versions before any code is written.

1. Project Overview & Architecture

1.1 Business objective

This system exists to solve two connected problems that together determine whether advertising spend generates profitable revenue.

Problem 1 — No early warning system: When ad creatives begin to fatigue, CTR declines gradually before collapsing. Without automated monitoring, this degradation goes undetected until it cascades through the entire funnel — from lower traffic to fewer inquiries to lost revenue. By the time the problem is visible, significant budget has been wasted.

Problem 2 — Blind optimization: Meta and TikTok ad platforms optimize for whatever signal they can measure. Without offline conversion data, they optimize for messages and inquiries — not for actual paid sales. This means ad budgets flow toward ads that generate cheap inquiries, not toward ads that generate revenue. High-value ads that attract real buyers may be starved of budget because their downstream revenue is invisible to the platform.

This system solves both problems. Part 3 catches creative degradation early — at the signal layer — before it becomes a revenue problem. Part 4 closes the attribution loop by feeding real sales data back to Meta, so the algorithm learns which ads actually drive revenue.

1.2 The 4-part system

Part	Name	Status	What it does
Part 1	Creative Production System	Not in this document	Defines how ad creatives are produced, structured, and supplied for testing
Part 2	Testing System	Not in this document	Defines how new creatives are tested before full deployment
Part 3	Analytics & Monitoring	INCLUDED — Section 2	Monitors ad performance metrics daily. Fires email alerts when CTR, CPC, frequency, or delivery deviate from established thresholds. Catches creative fatigue at the earliest signal layer.
Part 4	Conversion Tracking	INCLUDED — Section 3	Tracks every customer from ad click through WhatsApp / Messenger / Viber / Email contact, through quotation, through paid Sales Order — and sends that revenue data back to Meta CAPI so the algorithm optimizes for real buyers.

1.3 How Part 3 and Part 4 connect

Part 3 and Part 4 are independent systems that run in parallel, but they share data infrastructure and serve complementary functions:

- Both systems use Zendesk as a central data source. Part 3 monitors inquiry volume. Part 4 enriches tickets with ad tracking data.
- Both systems use Zoho Books. Part 3 monitors pipeline volume (estimates and sales orders created). Part 4 uses Sales Order creation as the CAPI conversion trigger.
- Both systems require Zendesk brand tagging. Set this up once — it serves both systems.
- Part 3 monitors the health of the funnel. Part 4 attributes the outcomes of that funnel back to specific ads.

1.4 Full system architecture

The complete data flow across both systems:

```
META ADS (ad click → fbclid generated)
         ↓
SHOPIFY (session token + fbclid captured in localStorage)
         ↓
CONTACT CHANNELS
  /wa-go  →  WhatsApp  →  Zendesk ticket (enriched with session token + fbclid)
  /msgr-go →  Messenger →  Zendesk ticket (enriched via PSID matching)
  /viber-go → Viber     →  Zendesk ticket (enriched via Viber user ID + time
window)
  Email link → Email    →  Zendesk ticket (token extracted via regex)
         ↓
ZENDESK (central inbox — all tickets tagged by brand + tracking data)
         ↓
n8n (automation engine — connects all systems)
  ├── RFQ detected → creates Estimate in Zoho Books → email notification
  └── Sales Order created → reads tracking fields → sends to Meta CAPI
         ↓
ZOHO BOOKS (Estimate → Sales Order → paid)
         ↓
META CAPI (receives: fbclid + sale amount + hashed email/phone)
         ↓
META ALGORITHM learns which ads generate real revenue

PARALLEL: ZOHO ANALYTICS monitors all data sources daily
  ├── Meta Ads connector (CTR, CPC, frequency, impressions)
```

```
   ├── Shopify connector (orders, sessions via GA4)
   ├── GA4 connector (traffic by source)
   ├── Zendesk connector (ticket volume by brand)
   └── Zoho Books connector (estimates and sales orders)
  → Fires email alerts when thresholds are breached
```

1.5 Multi-brand architecture

This system is designed for 10 or more brands running simultaneously. All brands share the same tech stack and the same target market. Zendesk is shared across all brands.

```
ARCHITECTURE MODEL:

One Zoho Analytics workspace per brand  →  One MASTER workspace (cross-brand view)
One n8n workflow set per brand (or shared with brand routing logic)
One set of Shopify tracking scripts per brand store
One Zendesk account — shared — with brand tagging per ticket
One Zoho Books organization per brand
One Meta ad account per brand
One Meta CAPI Dataset per brand
```

Workspace / Component	Purpose	Who accesses it
Primary brand workspace (Zoho Analytics)	Brand-level monitoring — all that brand's ads, traffic, inquiries, pipeline	That brand's marketing team + monitoring person
Brand B, C, D... workspaces	Same structure, one per brand	Each brand's marketing team + monitoring person
MASTER workspace (Zoho Analytics)	Cross-brand command center — all brands in one view	Monitoring person + business owner
Zendesk (shared)	All brands' inquiries in one inbox — filtered by Brand custom field	All brand teams + monitoring person
n8n (shared server)	Automation engine — handles all brands' workflow logic with brand routing	Developer only

1.6 Monitoring role and escalation structure

With 10+ brands running simultaneously, a single monitoring person watches the Master Dashboard and triages alerts. Brand teams execute fixes.

Role	What they do	What they do NOT do
Monitoring person (1)	Watches Master Dashboard daily. Receives all alerts across all brands. Triages by severity. Escalates to correct brand team.	Does not execute ad pauses, creative rotations, or audience changes.
Brand marketing team (per brand)	Receives escalated alerts for their brand. Executes the fix — pauses ad, rotates creative, expands audience. Reports resolution.	Does not monitor other brands.
Business owner	Receives CRITICAL alerts only. Reviews if multiple brands are simultaneously in critical state.	Not involved in routine monitoring.

```
Alert routing: WATCH alerts → monitoring person only.
WARNING alerts → monitoring person + relevant brand team.
CRITICAL alerts → monitoring person + brand team + business owner.
Every alert email subject line includes the brand name so recipients immediately
know which brand is affected.
```

1.7 Complete tech stack

Component	Technology	Role	Multi-brand note
Automation engine	n8n (self-hosted, Hetzner VPS)	Connects all systems. Handles webhooks, enrichment, and Meta CAPI calls.	One shared server, all brands
BI / Dashboard	Zoho Analytics	Central monitoring hub. All data lands here.	One workspace per brand + Master workspace
Meta Ads data	Zoho Analytics native connector	Pulls daily ad metrics	One connector per brand ad account
Shopify data	Zoho Analytics native connector	Pulls orders and product data	One connector per brand store
Session / traffic data	Google Analytics 4 + Zoho connector	Only way to get source-level traffic data	One GA4 property per brand store
Inquiry data	Zoho Analytics native	Pulls ticket volume	Shared Zendesk —

	connector (Zendesk)		filtered by brand tag
Revenue pipeline	Zoho Analytics native connector (Zoho Books)	Pulls estimates and sales orders	One connector per brand organization
CRM	Zendesk	Central inbox for all contact channels. Stores tracking fields.	Shared — brand custom field required
Accounting	Zoho Books	Quotations, estimates, sales orders. Triggers CAPI on SO creation.	One organization per brand
Tracking scripts	Shopify (theme.liquid + custom pages)	Captures fbclid and session token on ad click	One set per brand store — all on OS 2.0
Ad platform	Meta Ads (TikTok future state)	Receives offline conversion events via CAPI	One CAPI Dataset per brand
Alerting	Zoho Analytics built-in email alerts	Threshold-based email notifications	All alerts tagged with brand name
Server OS	Ubuntu 24.04 (Hetzner CX22)	Hosts n8n via Docker	One shared server
SSL / Proxy	Traefik (via Docker Compose)	Handles HTTPS for n8n	Built into n8n official Docker setup

1.8 Master build sequence

Follow this exact order. Do not start a step until all previous steps are verified complete.

```
STEP 0  — Upgrade all brand Shopify stores to Online Store 2.0 (BEFORE any code is
written)
STEP 1  — Set up n8n server on Hetzner (Section 3, Sprint 1)
STEP 2  — Set up Zoho Analytics connectors for primary brand (Section 2, Sprint 1)
STEP 3  — Set up Zendesk brand tagging (Section 2, Sprint 1 / Section 3, Sprint 3)
STEP 4  — Build Zoho Analytics dashboards for primary brand (Section 2, Sprint 2)
STEP 5  — Configure Zoho Analytics alerts for primary brand (Section 2, Sprint 3)
STEP 6  — Add Shopify tracking scripts for primary brand (Section 3, Sprint 2)
STEP 7  — Build n8n workflows (Section 3, Sprint 4)
STEP 8  — Set up Zoho Books custom fields and webhooks (Section 3, Sprint 5)
STEP 9  — Configure Meta CAPI Dataset and test (Section 3, Sprint 6)
```

```
STEP 10 — Verify full end-to-end funnel for primary brand
STEP 11 — Replicate for all other brands (Section 2 Sprint 4 / Section 3 Sprint 7)
STEP 12 — Build Master Dashboard in Zoho Analytics (Section 2, Sprint 4)
STEP 13 — Run full system acceptance checklist
```

1.9 Key decisions log

These decisions are final. Do not re-evaluate them during development.

Decision	What was decided	Why
Automation engine	n8n self-hosted on Hetzner VPS	Flat cost at scale (~€4/month) vs Make.com ($59+/month at high volume). All brands share one server.
SSL proxy	Traefik (not Nginx + Certbot)	Official n8n recommendation. Simpler config. Included in n8n Docker Compose.
Notifications	Email only	No Viber bot, no Zoho Cliq. Email is reliable, zero extra infrastructure, works for all recipients.
Ad platform	Meta only	TikTok is future state. Architecture supports TikTok without rebuild — just activate when ready.
Shopify theme version	Online Store 2.0 only	Redirect pages require OS 2.0 JSON templates. Upgrade all stores before dev starts.
CAPI trigger	Sales Order created in Zoho Books	Staff manually converts paid quotation to Sales Order. This is the confirmed business process.
Viber attribution	Time-based first contact + Viber user ID for repeat contacts	Oapps does not expose phone number. Viber user ID is stored and available for repeat matching.
Zendesk	Shared across all brands with Brand custom field	One Zendesk account. Every ticket auto-tagged with brand via triggers per channel.
Zoho Books webhook	Outgoing webhook via Workflow Rule	No native n8n Zoho Books node exists. Zoho Books fires POST to n8n webhook URL on Sales Order creation.
Meta Offline API	CAPI Datasets (not Offline Conversions API)	Meta permanently discontinued Offline Conversions API on May 14, 2025. Must use standard CAPI.
Zoho region	US datacenter	Philippines accounts default to US datacenter. Auth URL: accounts.zoho.com

Multi-brand scope	All brands from the start	Not phased by brand. Build primary brand first as template, then replicate.

2. Part 3: Analytics & Monitoring System

2.1 System purpose

Part 3 is an automated funnel monitoring system. It connects five data sources — Meta Ads, Shopify, GA4, Zendesk, and Zoho Books — into Zoho Analytics, and sends email alerts when key metrics fall below defined thresholds.

The system catches problems at the earliest possible layer. A creative fatigue event that begins as a CTR decline in Meta Ads will, if undetected, progress to a traffic drop visible in GA4/Shopify, then to an inquiry volume drop in Zendesk, and finally to a pipeline drop in Zoho Books. The alert system is designed to catch the signal at layer 1 — not layer 3 or 4.

Layer	Platform	Signal type	Detection timing
1	Meta Ads	CTR, CPC, impressions, frequency per ad and adset	Earliest — catch creative fatigue here before it hits traffic
2	GA4 / Shopify	Sessions by traffic source (facebook/paid_social)	Middle — confirms ad degradation is hitting site traffic
3	Zendesk	Daily ticket volume, inquiry rate vs sessions	Late — damage already started on inquiries
4	Zoho Books	Estimates and sales orders created per period	Lagging — revenue impact now confirmed

2.2 Credentials required for Part 3

Collect all credentials below before starting Sprint 1. Items marked * must be collected per brand — start with primary brand only.

Credential	Where to get it	Multi-brand?	Used in
Zoho Analytics admin login	From client	One login, all workspaces	All sprints
Zoho Books admin login (same Zoho account)	From client	One login, all organizations	Sprint 1
Meta Business Manager admin access	From client	One BM, all ad accounts	Sprint 1

Meta Ad Account ID *	Meta Business Manager → Ad Accounts	One per brand	Sprint 1
Shopify Partner account	partners.shopify.com	One account, all stores	Sprint 1
Shopify Client ID + Client Secret *	Created during Shopify Partner app setup	One app per brand store	Sprint 1
Shopify store domain * ([store].myshopify.com)	From client	One per brand	Sprint 1
Zendesk subdomain + admin login	From client	Shared — one login	Sprint 1
Zendesk brand values list	From client — exact brand names as they should appear in dropdown	Required for brand tagging	Sprint 1
Google account with GA4 access	From client	One account, all properties	Pre-sprint
GA4 Measurement ID * (G-XXXXXXXX)	GA4 Admin → Data Streams → Web	One per brand store	Pre-sprint

2.3 Pre-Sprint: GA4 setup

GA4 must be installed and collecting data before the Zoho Analytics connector is set up. The Shopify native connector does not expose session-level traffic source data — GA4 is the only way to see how many sessions came from facebook/paid_social vs organic vs direct.

Install GA4 on each brand Shopify store

1. Open Google Analytics (analytics.google.com). Create a new GA4 property for the brand if one does not exist.
2. In the property, go to Admin → Data Streams → Add stream → Web → enter the brand's Shopify store URL → Create stream. Copy the Measurement ID (G-XXXXXXXX).
3. In Shopify admin: Online Store → Themes → [active theme] → ⋯ → Edit code → Layout folder → theme.liquid.
4. Paste the GA4 gtag snippet immediately before the </head> closing tag. Replace G-XXXXXXXX with the actual Measurement ID.
5. Click Save.
6. Verify: Open the brand's store in a browser. In GA4 → Reports → Realtime — confirm a session appears. This confirms the tag is firing.

Confirm UTM parameters on all Meta ads

GA4 will only classify Meta ad traffic correctly if UTM parameters are present on every ad's destination URL. Without UTMs, Meta traffic appears as direct or referral — invisible to source-level analysis.

1. In Meta Ads Manager, open each active campaign.
2. For every ad, open the ad creative and check the Website URL field.
3. Confirm the URL contains at minimum: ?utm_source=facebook&utm_medium=paid_social&utm_campaign=[campaign_name]
4. Add UTMs to any ad missing them. This is required before the GA4 connector will show meaningful data.
5. Going forward: add UTM parameters to every new ad destination URL before the campaign goes live.

```
Note: Zoho Analytics also uses a native Meta Ads connector that pulls impression,
click, CTR, and CPC data
directly from the Meta API. GA4 is used exclusively for session and traffic source
data.
Both are required. GA4 does not replace the Meta Ads connector — they serve
different purposes.
```

2.4 Sprint 1: Connect all data sources

Estimated time: 1 day per brand. Build all connectors for the primary brand first. After full verification, repeat for each additional brand.

```
Build order within this sprint:
1. Meta Ads connector   2. Shopify connector   3. GA4 connector
4. Zendesk connector + brand tagging   5. Zoho Books connector
6. Verify all connections
```

2.4.1 Meta Ads connector

Connects Meta ad account to Zoho Analytics. Pulls daily snapshots of ad-level and adset-level performance metrics. This connector is bundled free with all paid Zoho Analytics plans.

1. Log in to Zoho Analytics. On the home page, click Import Your Data.
2. In the search bar or connector list, find and select Facebook Ads Insight Analytics.
3. In the setup dialog, enter a Workspace Name: [Brand Name] — Ads Monitoring.
4. Click Authenticate Facebook Ads. A Meta login window will appear. Log in with the Meta Business Manager admin account.

5. After authentication, select the brand's Ad Account from the dropdown.
6. Select the following modules to sync: Accounts, Ad Account Insights, Campaigns, Campaign Insights, Ad Sets, Ad Set Insights, Ads, Ad Insights.
7. Set sync frequency to Every day.
8. Click Create. Initial data import may take up to 2 hours depending on account size. Zoho Analytics will send an email when complete.
9. After import completes: click Data Sources in the left panel. Confirm the Meta Ads connector shows a green status and lists a recent sync time.

```
Important — which table to use for alerts:
Ad Insights = ad-level data (CTR, CPC per individual ad). Use for Creative Fatigue
and Underperforming Ads alerts.
Ad Set Insights = adset-level data (Frequency). Use for Audience Saturation
alerts.
Campaign Insights = campaign-level data (Impressions, Spend). Use for Delivery
Problems alerts.
The actual table names in Zoho Analytics may differ slightly. Check Data Sources →
table list for exact names
before writing any Query Tables or alert conditions.
```

2.4.2 Shopify connector

Connects Shopify store to Zoho Analytics. Requires creating a custom app in Shopify Partners. This is a one-time setup per brand store.

Step A — Create Shopify Partner app:

1. Log in to partners.shopify.com. From the left navigation, select App distribution → All Apps.
2. Click Visit Dev Dashboard.
3. Click Create App. Enter an app name (e.g. 'Zoho Analytics Connector — [Brand Name]'). Click Create.
4. Navigate to the Access section. Under Scopes, select all of the following: read_orders, read_products, read_customers, read_draft_orders, read_locations, read_fulfillments, read_price_rules, read_marketing_events, read_shopify_payments_payout, read_shopify_payments_disputes, read_shopify_payments_accounts, read_discounts.
5. Open Zoho Analytics in a separate browser tab. Go to Import Your Data → E-Commerce tab → Shopify. Copy the Redirect URL shown in the Step 1 dialog.
6. Return to the Shopify Dev Dashboard. Paste the Redirect URL into the Redirect URL field. Click Release.
7. From the app's Home tab, navigate to Distribution → Select Distribution Method → choose Custom Distribution.
8. Enter the brand's store domain (e.g. homecartel.myshopify.com). Ensure 'Allow multi-store install' is unchecked. Click Generate Link.
9. Copy the Install link. Open it in a new tab. Click Install to install the app on the store.
10. Return to the Shopify Dev Dashboard → Settings tab. Copy the Client ID and Client Secret.

Step B — Connect in Zoho Analytics:

1. In Zoho Analytics: Import Your Data → E-Commerce tab → Shopify.
2. Enter a Workspace Name: [Brand Name] — Store Data. Enter the store domain, Client ID, and Client Secret from Step A.
3. Click Authenticate Shopify. After redirect, click Update to complete authentication.
4. Select modules to sync: Shop, Customer, Order, Order Line Items, Product, Product Variants, Draft Orders, Abandoned Checkouts, Fulfillments, Refunds.
5. In Fetch data period: select All time (or at minimum Last 90 days). Note: to import data older than 90 days, the Read all orders scope must be requested separately — see below.
6. Set sync frequency to Every day. Click Create.
7. Wait for initial import email confirmation.

```
To import order data older than 90 days:
In the Shopify Dev Dashboard, open the app → Versions → Access section → click
Request access.
Locate Read all orders scope → click Request access → provide a reason → submit.
Once approved, select the scope, save the app, then re-authenticate in Zoho
Analytics
and click Full Data Fetch to retrieve all historical order data.
```

2.4.3 GA4 connector

Connects Google Analytics 4 to Zoho Analytics. Provides session and traffic source data that the Shopify connector does not expose.

1. In Zoho Analytics: Import Your Data → search for Google Analytics 4 or find it under the Analytics category.
2. Click Connect. Authenticate with the Google account that has GA4 access for this brand.
3. Select the brand's GA4 Property.
4. Select dimensions and metrics to import. At minimum: Date, Session source, Session medium, Sessions, Users, Bounce rate, Average session duration.
5. Set sync frequency to Every day. Click Create.
6. After import: verify the sessions table contains rows with source = 'facebook' and medium = 'paid_social'. If these are not present, UTM parameters are not correctly set on Meta ads (return to Pre-Sprint step).

2.4.4 Zendesk connector + brand tagging

The Zendesk connector pulls ticket volume data. Since Zendesk is shared across all brands, every ticket must carry a brand identifier before the connector can filter by brand. Set up brand tagging first, then connect.

Step A — Create Brand custom field in Zendesk:

1. Log in to Zendesk. Go to Admin Center (gear icon, top right).
2. Navigate to Objects and rules → Tickets → Fields.
3. Click Add field. Select Dropdown.
4. Field display name: Brand. Important: do NOT name this field 'channel' — that is a reserved Zendesk system name.
5. Add dropdown values — one per brand. Example: Primary Brand, Brand B, Brand C, Brand D. Add all current brands. New brands can be added later.
6. Under Field agents can edit: enable Required to solve ticket. This forces agents to tag every ticket before closing.
7. Click Save.

Step B — Create auto-tagging triggers per brand per channel:

Each brand has distinct contact channels (different WhatsApp number, Messenger page, email address). Zendesk auto-populates the Brand field based on which channel the ticket came from.

1. Go to Admin Center → Objects and rules → Triggers → Add Trigger.
2. Create one trigger per brand per channel. The trigger detects incoming channel and sets the Brand field automatically.

```
TRIGGER TEMPLATE — Example for Primary Brand WhatsApp:

Trigger name: Auto-tag Brand — Primary Brand — WhatsApp

Conditions (Meet ALL):
  Ticket > Is > Created
  Ticket > Channel > Is > WhatsApp
  [Add condition for specific WhatsApp number if multiple brands share WhatsApp
Business platform]

Actions:
  Ticket > [Brand field] > Primary Brand
  Ticket > Add tags > brand_primary

CHANNEL CONDITIONS (exact values from Zendesk trigger dropdown):
  WhatsApp tickets:   Ticket > Channel > Is > WhatsApp   (under Social messaging
group)
  Messenger tickets:  Ticket > Channel > Is > Facebook Messenger   (under Social
messaging group)
```

```
  Viber tickets:      Ticket > Channel > Is > Viber   (under Sunshine
Conversations group)
  Email tickets:      Ticket > Channel > Is > Email

Create one trigger per brand per channel. Names example:
  Auto-tag Brand — Brand B — WhatsApp
  Auto-tag Brand — Brand B — Messenger
  Auto-tag Brand — Brand B — Viber
  Auto-tag Brand — Brand B — Email
```

3. After all brand triggers are created, add a fallback trigger for untagged tickets:

```
FALLBACK TRIGGER:

Trigger name: Flag Untagged Brand
Conditions: Ticket > Is > Created  AND  [Brand field] > Is > (blank)
Actions: Add tags > brand_untagged
         Assign to [monitoring person or brand ops queue]
```

Step C — Connect Zendesk in Zoho Analytics:

1. In Zoho Analytics: Import Your Data → search for Zendesk or find it under the Support category.
2. Click Connect. Authenticate with the Zendesk subdomain and admin login.
3. Select modules: Tickets, Users, Organizations, Groups, Ticket Metrics.
4. Set sync frequency to Every day. Click Create.
5. After import: open the Tickets table. Confirm the Brand custom field column is present and populated on recent tickets. Check: filter by brand_untagged tag — if this returns many tickets, the auto-tagging triggers need adjustment.

2.4.5 Zoho Books connector

1. In Zoho Analytics: Import Your Data → Zoho Apps → Zoho Books.
2. Since this is the same Zoho account, authentication is automatic. Select the primary brand's organization from the dropdown.
3. Select modules: Estimates, Sales Orders, Invoices, Contacts, Items.
4. Set sync frequency to Every day. Click Create.
5. Repeat for each additional brand's Zoho Books organization — one connector per brand.

2.4.6 Verify all connections — Sprint 1 acceptance

Source	Check	Expected result
Meta Ads	Open Ad Insights table in Zoho Analytics	Rows with CTR, CPC, Impressions, Frequency, Amount Spent columns. Dates within last 7 days.
Shopify	Open Orders table	Order records with Created At dates, amounts, Customer IDs. Recent orders visible.
GA4	Open sessions table	Rows with source/medium breakdown. 'facebook / paid_social' and 'instagram / paid_social' visible as sources.
Zendesk	Open Tickets table. Filter by Brand field.	Ticket records with Brand field populated. brand_untagged count is low (ideally zero).
Zoho Books	Open Estimates table AND Sales Orders table	Records present in both tables. Created At dates visible.

```
Sprint 1 is complete when ALL of the following are true:
 - All 5 connectors show green status in Zoho Analytics Data Sources tab
 - Each connector has completed at least one successful sync
 - GA4 sessions table shows facebook/paid_social and instagram/paid_social as
traffic sources
 - Zendesk Tickets table has Brand field populated on recent tickets
 - brand_untagged count in Zendesk is at or near zero
 - Zoho Books returns both Estimates and Sales Orders tables with real data
 - No sync failures or authentication errors in any connector
```

2.5 Sprint 2: Build dashboards

Estimated time: 2-3 days per brand. Build all 4 brand-level dashboard views and the Master Dashboard cross-brand views. Dashboards are built for the primary brand first — the same structure is duplicated for each additional brand in Sprint 4.

```
Dashboard design principle: Build with a Brand filter from the start. Even if only
one brand is
connected now, the Brand filter dropdown must be present in every dashboard.
Adding new brands
later will then require zero dashboard redesign — only new data.
```

```
Before building any dashboard, build the Inquiry Rate Daily Query Table (section
```

```
2.5.1).
Dashboards that show inquiry rate will depend on this Query Table. Build it first.
```

2.5.1 Inquiry Rate Daily Query Table (build before dashboards)

The Inquiry Rate requires joining Zendesk ticket counts with GA4 session counts by date. This cannot be done with a standard chart — it requires a Query Table.

To create a Query Table in Zoho Analytics: In your workspace, click + Create → Query Table. Enter the query below. Name it 'Inquiry Rate Daily'. Click Save.

```
SELECT
    z.date AS date,
    z.brand AS brand,
    z.ticket_count AS daily_tickets,
    g.sessions AS daily_sessions,
    ROUND((z.ticket_count * 1.0 / NULLIF(g.sessions, 0)) * 100, 2) AS
inquiry_rate_pct
FROM
    (SELECT DATE(created_at) as date, brand, COUNT(*) as ticket_count
     FROM Tickets
     WHERE brand IS NOT NULL
     GROUP BY DATE(created_at), brand) z
LEFT JOIN
    (SELECT date, SUM(sessions) as sessions
     FROM GA4_Sessions
     GROUP BY date) g
    ON z.date = g.date
ORDER BY date DESC, brand ASC

Note: Replace 'Tickets' and 'GA4_Sessions' with the exact table names in your
workspace.
Check Data Sources → table list for exact names before running.
```

2.5.2 Dashboard View 1 — Ad Performance (daily)

Widget	Chart type	Data source	Configuration
CTR trend (last 30 days)	Line chart	Ad Insights	X-axis: date. Y-axis: CTR (Link Click-Through Rate). Filter: status = ACTIVE. Brand filter applied.

CPC trend (last 30 days)	Line chart	Ad Insights	X-axis: date. Y-axis: CPC. Filter: status = ACTIVE. Brand filter applied.
Top 10 ads by CTR (last 7 days)	Table	Ad Insights	Columns: ad name, campaign name, CTR, CPC, impressions, spend. Sorted CTR descending. Last 7 days.
Bottom 10 ads by CTR (last 7 days)	Table	Ad Insights	Same columns as above. Sorted CTR ascending. Last 7 days. Highlights underperformers.
Total spend by campaign (weekly)	Bar chart	Campaign Insights	X-axis: campaign name. Y-axis: amount spent. Last 7 days. Brand filter applied.

2.5.3 Dashboard View 2 — Traffic & Sessions (daily)

Widget	Chart type	Data source	Configuration
Sessions by source/medium (last 30 days)	Line chart	GA4 Sessions	X-axis: date. Y-axis: sessions. Series: facebook/paid_social, instagram/paid_social, organic, direct.
Paid social sessions vs total sessions (weekly)	Bar chart	GA4 Sessions	Stacked bar. Compare paid social share vs all other sources. Shows reliance on paid traffic.
Sessions from paid social (30-day trend)	KPI widget	GA4 Sessions	Single number showing WoW % change. Green if up, red if down.

2.5.4 Dashboard View 3 — Inquiry Health (daily)

Widget	Chart type	Data source	Configuration
Daily ticket volume (last 30 days)	Line chart	Zendesk Tickets	X-axis: date. Y-axis: ticket count. Brand filter applied.
Inquiry rate % (last 30 days)	Line chart	Inquiry Rate Daily Query Table	X-axis: date. Y-axis: inquiry_rate_pct. Brand filter applied.
Tickets by channel (last 30 days)	Pie or donut chart	Zendesk Tickets	Breakdown by channel: WhatsApp, Messenger, Viber, Email.
Tickets by brand (weekly)	Bar chart	Zendesk Tickets	All brands side by side. Shows which brands are receiving inquiries.
brand_untagged ticket count	KPI widget	Zendesk Tickets	Single number. If > 0, brand tagging system needs attention. Should stay near zero.

2.5.5 Dashboard View 4 — Revenue Pipeline (daily)

Widget	Chart type	Data source	Configuration
Estimates created (last 30 days)	Line chart	Zoho Books Estimates	X-axis: date. Y-axis: count of estimates created. Brand filter applied.
Sales orders created (last 30 days)	Line chart	Zoho Books Sales Orders	X-axis: date. Y-axis: count of sales orders created. Brand filter applied.
Estimate to Sales Order conversion rate	KPI widget	Zoho Books Estimates + Sales Orders	Ratio: SOs created / Estimates created in same period. Trend indicator.
Open estimates (current)	Table	Zoho Books Estimates	List of estimates with status = Open or Sent. Customer name, amount, created date.

2.5.6 How to build each dashboard in Zoho Analytics

1. In the brand workspace, click + Create → Dashboard.
2. Name the dashboard (e.g. 'Ad Performance — [Brand Name]').
3. Click Add Widget. Select Chart, Table, or KPI as needed.
4. Select the data source table (Meta Ads connector table, GA4 table, Zendesk Tickets, etc).
5. Configure axes, filters, and date ranges as specified in the widget tables above.
6. Add a Brand filter: click the filter icon in the dashboard → Add Filter → select the Brand field → set default value to this brand's name.
7. Save the dashboard.
8. Repeat for all 4 dashboard views.

2.6 Sprint 3: Configure email alerts

Estimated time: 1 day per brand. All alert thresholds are confirmed from 16 months of real account data (January 2025 to May 2026), covering 2,284 active performance ads. No thresholds are pending — configure all alerts immediately.

2.6.1 FR Stores campaign exclusion — critical

```
REQUIRED: Exclude the FR Stores Reach campaign from ALL alert queries and alert
conditions.

The FR Stores campaign uses a Reach objective, not a Traffic or Sales objective.
```

```
CTR for Reach campaigns is naturally 0.03-0.06% by design — this is not
underperformance.

Add this filter to every Query Table WHERE clause:
  AND campaign_name NOT LIKE '%FR Stores%'

AND set this condition in every alert that uses ad-level data:
  Campaign name does not contain 'FR Stores'

Failure to exclude this campaign will cause constant false Critical alerts.
```

2.6.2 Build Query Tables A, B, C before configuring alerts

Three alerts (Creative Fatigue, Expensive Ads, Delivery Problems) require calculated fields that Zoho Analytics cannot compute from raw connector data alone. Build these Query Tables first, then build alerts on top of them.

Query Table A — Creative Fatigue Score (CTR this week vs prior week per ad):

```
SELECT
    ad_name,
    campaign_name,
    adset_name,
    ROUND(AVG(CASE WHEN date >= DATE_SUB(TODAY(), INTERVAL 7 DAY)
               THEN ctr_link_click_through_rate END), 2) AS ctr_last_7d,
    ROUND(AVG(CASE WHEN date >= DATE_SUB(TODAY(), INTERVAL 14 DAY)
               AND date < DATE_SUB(TODAY(), INTERVAL 7 DAY)
               THEN ctr_link_click_through_rate END), 2) AS ctr_prior_7d,
    ROUND(((ctr_last_7d - ctr_prior_7d) / NULLIF(ctr_prior_7d, 0)) * 100, 1) AS
ctr_change_pct
FROM Ad_Insights
WHERE date >= DATE_SUB(TODAY(), INTERVAL 14 DAY)
  AND status = 'ACTIVE'
  AND campaign_name NOT LIKE '%FR Stores%'
GROUP BY ad_name, campaign_name, adset_name
ORDER BY ctr_change_pct ASC

Replace 'Ad_Insights' and column names with exact names from your workspace
tables.
CTR column may be named 'CTR (Link Click-Through Rate)' — verify in table schema
first.
```

Query Table B — CPC Efficiency Score (CPC this week vs 30-day average per ad):

```
SELECT
    ad_name,
    campaign_name,
    ROUND(AVG(CASE WHEN date >= DATE_SUB(TODAY(), INTERVAL 7 DAY)
               THEN cpc_cost_per_link_click END), 2) AS cpc_last_7d,
    ROUND(AVG(CASE WHEN date >= DATE_SUB(TODAY(), INTERVAL 30 DAY)
               THEN cpc_cost_per_link_click END), 2) AS cpc_30d_avg,
    ROUND(((cpc_last_7d - cpc_30d_avg) / NULLIF(cpc_30d_avg, 0)) * 100, 1) AS
cpc_rise_pct
FROM Ad_Insights
WHERE date >= DATE_SUB(TODAY(), INTERVAL 30 DAY)
  AND status = 'ACTIVE'
  AND campaign_name NOT LIKE '%FR Stores%'
GROUP BY ad_name, campaign_name
ORDER BY cpc_rise_pct DESC
```

Query Table C — Delivery Health Score (impressions WoW vs spend WoW per campaign):

```
SELECT
    campaign_name,
    SUM(CASE WHEN date >= DATE_SUB(TODAY(), INTERVAL 7 DAY)
             THEN impressions END) AS imp_this_week,
    SUM(CASE WHEN date >= DATE_SUB(TODAY(), INTERVAL 14 DAY)
             AND date < DATE_SUB(TODAY(), INTERVAL 7 DAY)
             THEN impressions END) AS imp_last_week,
    SUM(CASE WHEN date >= DATE_SUB(TODAY(), INTERVAL 7 DAY)
             THEN amount_spent END) AS spend_this_week,
    SUM(CASE WHEN date >= DATE_SUB(TODAY(), INTERVAL 14 DAY)
             AND date < DATE_SUB(TODAY(), INTERVAL 7 DAY)
             THEN amount_spent END) AS spend_last_week,
    ROUND(((imp_this_week - imp_last_week) / NULLIF(imp_last_week,0))*100,1) AS
imp_wow_pct,
    ROUND(((spend_this_week - spend_last_week) / NULLIF(spend_last_week,0))*100,1)
AS spend_wow_pct
FROM Campaign_Insights
WHERE date >= DATE_SUB(TODAY(), INTERVAL 14 DAY)
  AND campaign_name NOT LIKE '%FR Stores%'
GROUP BY campaign_name
```

2.6.3 Alert thresholds — confirmed from historical data

All thresholds below are derived from 16 months of real account data (January 2025 to May 2026), 2,284 active performance ads excluding FR Stores. These are not industry benchmarks — they reflect this account's actual performance distribution.

Metric	10th pct	25th pct	Median	75th pct	90th pct	Threshold basis
CTR — performance ads	0.43%	0.67%	1.17%	1.84%	2.59%	Warning = 25th pct | Critical = 10th pct
Frequency — adsets	—	—	2.37	3.71	4.46	Watch = above median | Critical = 90th pct
CPC	₱2.50	₱4.45	₱9.15	₱16.31	₱26.48	% rise above own 30-day average

```
Key insight from data: Carousel with Cover ads significantly outperform Single
Image ads.
Carousel median CTR: 1.33% vs Single Image median CTR: 0.65%.
Carousel median CPC: ₱5.53 vs Single Image median CPC: ₱15.61 (nearly 3x higher).
When underperforming alerts fire, check ad format first — Single Image ads are the
most
common source of low-CTR alerts in this account.
```

2.6.4 Complete alert configuration table

Configure all 11 alerts below. How to create an alert: open the relevant dashboard view or Query Table → click the Alert icon (bell) → Create Alert → enter name → set conditions → set How often → select Email → configure recipients → select Include Report → Save.

Level	Parameter	Trigger condition	Required action	Source
WATCH	Creative Fatigue	CTR dropped 20-29% vs prior 7 days on same active ad	Flag for review at next weekly meeting	Query Table A
WATCH	Audience Saturation	Frequency > 2.5 on any adset (7-day window). Historical median: 2.37	Monitor closely. Note adset name.	Ad Set Insights
WARNING	Creative Fatigue	CTR dropped 30-44% vs prior 7 days on same active ad	Review ad immediately. Prepare replacement creative.	Query Table A
WARNING	Underperforming Ads	CTR < 0.67% on any active performance ad (last 7 days). Historical 25th percentile.	Review creative. Plan rotation if no improvement in 3 days.	Ad Insights

WARNING	Expensive Ads	CPC risen 30-49% above the ad's own 30-day average	Review targeting and creative. Plan budget reallocation.	Query Table B
WARNING	Audience Saturation	Frequency > 3.5 on any adset (7-day). Historical 75th percentile: 3.71	Expand audience OR rotate creative on this adset.	Ad Set Insights
CRITICAL	Creative Fatigue	CTR dropped 45%+ vs prior 7 days on same active ad	Pause ad immediately. Rotate creative same day.	Query Table A
CRITICAL	Underperforming Ads	CTR < 0.43% on any active performance ad (last 7 days). Historical 10th percentile.	Pause ad. Do not continue spending.	Ad Insights
CRITICAL	Expensive Ads	CPC risen 50%+ above the ad's own 30-day average	Pause ad. Reallocate budget to better-performing ads.	Query Table B
CRITICAL	Audience Saturation	Frequency > 4.5 on any adset (7-day). Historical 90th percentile: 4.46	Pause adset immediately. Audience burned. Expand or replace.	Ad Set Insights
CRITICAL	Delivery Problem	Impressions dropped 25%+ WoW while spend dropped less than 10% on same campaign	Check ad account health, payment, and policy flags in Meta Business Manager.	Query Table C

2.6.5 Alert email format

Configure every alert with the brand name in the subject line. This is critical — with 10+ brands, recipients must immediately know which brand triggered the alert.

```
Subject line format:   [LEVEL] — [Brand Name] — [Parameter] — [Date]
Example:               CRITICAL — Primary Brand — Creative Fatigue — 2026-07-14

Email body: Include the specific ad or adset name, the metric values (e.g. CTR
this week vs last week),
the threshold that was breached, and the required action. Select 'Include Report'
and attach the
relevant dashboard view as a PDF so recipients can see the data immediately.

Alert routing:
  WATCH   → monitoring person only
  WARNING → monitoring person + relevant brand marketing team
```

```
CRITICAL → monitoring person + brand team + business owner
```

2.6.6 Daily digest email

In addition to threshold-based alerts, configure a daily digest email for each brand workspace. This gives the monitoring person a daily snapshot even when no alerts fire.

1. Open any dashboard in the brand workspace.
2. Click Share → Email.
3. Select format: HTML (inline in email).
4. Check Schedule this Email.
5. Set interval: Daily. Set time: 08:00 (or desired local morning time).
6. Add recipients: monitoring person + business owner.
7. Enter Schedule name: Daily Digest — [Brand Name].
8. Click Save Email Schedule.
9. Repeat for each brand workspace.

```
Sprint 2 & 3 are complete when ALL of the following are true:
 - All 4 dashboard views built and showing real data for primary brand
 - Inquiry Rate Daily Query Table returns rows with non-null inquiry_rate_pct
values
 - Query Tables A, B, C built and returning correct calculated data
 - FR Stores exclusion filter confirmed in all 3 Query Tables
 - All 11 alerts configured and active (check Settings → Alerts section in
workspace)
 - Brand name appears in subject line of all alerts
 - Alert routing confirmed: Watch, Warning, Critical go to correct recipient
groups
 - Daily digest email sent and received successfully at least once
 - Test alert triggered for each severity level — correct recipients received it
```

2.7 Sprint 4: Multi-brand rollout & Master Dashboard

Estimated time: 1-2 days per additional brand + 2-3 days for Master Dashboard. Execute after primary brand instance is fully verified and signed off.

```
Do not start Sprint 4 until Sprints 1-3 for the primary brand are signed off.
Each new brand is onboarded by duplicating the template. Build one brand at a
time.
```

2.7.1 Onboarding each additional brand

1. Create a new Zoho Analytics workspace: [Brand Name] — Ads Monitoring.
2. Connect Meta Ads connector — select this brand's ad account ID.
3. Set up Shopify Partner app for this brand's store and connect Shopify connector.
4. Connect GA4 connector — select this brand's GA4 property.
5. Connect Zendesk connector — same account, same subdomain. Filter all views by Brand = [Brand Name].
6. Connect Zoho Books connector — select this brand's organization.
7. Build all 4 dashboard views — identical structure to primary brand, Brand filter pre-set.
8. Build Query Tables A, B, and C — add brand filter to WHERE clause: AND brand = '[Brand Name]'.
9. Configure all 11 alerts with brand name in subject line.
10. Configure daily digest email.
11. Verify all data flowing and alerts firing correctly before moving to next brand.

```
Threshold note: The confirmed thresholds (CTR < 0.67%, Frequency > 2.5, etc.) are
derived from
the primary brand's 16 months of historical data. Apply these same thresholds to
all brands as
the starting point. After 60 days of data for each new brand, run the same
historical analysis
and recalibrate if the brand's performance profile differs significantly.
```

2.7.2 Master Dashboard — cross-brand command center

The Master Dashboard is a separate Zoho Analytics workspace that the monitoring person watches daily. It shows all brands in one view.

Master Dashboard View 1 — Brand health scorecard:

Widget	Data	Configuration
Brand health scorecard table	All brand workspaces	Table: one row per brand. Columns: Brand name, CTR status (green/amber/red), Sessions status, Ticket volume status, Pipeline status. Color-coded by threshold.

Active alerts feed (last 24h)	All brand alert logs	List of all alerts fired in last 24 hours. Sorted by severity. Tagged by brand name.
Total spend by brand (weekly)	All Meta ad accounts	Bar chart — spend per brand side by side. Shows budget distribution across portfolio.
Total inquiries by brand (weekly)	Zendesk — all brands	Bar chart — ticket count per brand. Shows which brands are generating inquiry volume.

Master Dashboard View 2 — Cross-brand ad performance:

Widget	Data	Configuration
CTR by brand (8-week trend)	All Meta accounts	Line chart — one line per brand. Shows relative CTR performance across portfolio.
CPC by brand (8-week trend)	All Meta accounts	Line chart — one line per brand.
Worst performing ads across all brands	All Meta accounts	Table — CTR ranked ascending, brand column included, last 7 days. Instant portfolio-level view of at-risk ads.

Portfolio-level alerts — configure in Master workspace:

Alert	Trigger condition	Recipients
Brand offline	Any brand has zero ad impressions for 3 consecutive days	Monitoring person + business owner
Portfolio CTR decline	Average CTR across ALL brands drops 20%+ week-over-week	Monitoring person + business owner
Multiple brands critical	3 or more brands have active CRITICAL alerts simultaneously	Monitoring person + business owner
Untagged ticket spike	brand_untagged ticket count exceeds 20 in any 24-hour period	Monitoring person — brand tagging system needs attention

```
Sprint 4 is complete when ALL of the following are true:
 - All intended brands have been onboarded with their own workspace
 - Each brand workspace has all 5 connectors syncing and all 4 dashboards built
 - Each brand has all 11 alerts configured and active
 - Zendesk brand tagging is working for all brands — brand_untagged count near
zero
 - Master Dashboard workspace is built with all 4 cross-brand views
 - Portfolio-level alerts are configured and tested in Master workspace
```

```
- Monitoring person has been briefed on the two-tier escalation structure
- Each brand team knows which alerts they are responsible for acting on
```

3. Part 4: Conversion Tracking System

3.1 System purpose

Part 4 solves a fundamental attribution problem: the sales process happens offline, but Meta only sees online clicks. Without this system, Meta optimizes for cheap inquiries rather than real paid sales. High-value ads that attract genuine buyers may be starved of budget because their downstream revenue is invisible to the platform.

This system creates a complete closed-loop tracking pipeline:

```
Ad Click (Meta generates fbclid)
      ↓
Shopify (captures fbclid + generates session token → stored in browser
localStorage)
      ↓
Customer clicks contact button → /wa-go or /viber-go or /msgr-go redirect page
      (page logs session token + fbclid to n8n webhook in <200ms, then opens
channel)
      ↓
Zendesk ticket created (n8n enriches ticket with session token + fbclid + brand +
channel)
      ↓
Customer requests quotation → n8n detects RFQ keywords → creates Estimate shell in
Zoho Books
      (estimate already contains session token + fbclid + tracking fields)
      ↓
Sales agent sends quotation → customer pays → staff converts to Sales Order in
Zoho Books
      ↓
Zoho Books fires webhook to n8n (Sales Order created event)
      ↓
n8n reads custom fields (session token, fbclid, sale amount, customer email/phone)
n8n normalizes + SHA-256 hashes email and phone
n8n sends event to Meta CAPI: action_source=system_generated, value=sale amount,
fbclid
      ↓
Meta learns which ad clicks, audiences, and creatives generate real paid revenue
```

3.2 Session token design

The session token is the single identifier that links all systems together. It is generated on Shopify when a customer arrives from any source, and must flow through every subsequent system without being lost or overwritten.

Property	Value
Format	HC-{timestamp}-{random5digit} Example: HC-20260714-18291
Generated by	JavaScript in Shopify theme.liquid on every page load
Stored in	Browser localStorage (key: session_token). Also: fbclid, utm_source, utm_medium, utm_campaign, utm_content stored separately.
Passed through	Shopify redirect pages → n8n webhook → Zendesk custom field → Zoho Books estimate custom field → Zoho Books sales order custom field → Meta CAPI event_id
Expiry	localStorage persists until cleared by user or browser. Safari clears after 7 days of inactivity (known limitation).
Collision prevention	Timestamp (milliseconds) + 5-digit random number makes collision probability negligible
Uniqueness rule	Never overwrite an existing session_token on a Zendesk ticket. If token already exists, preserve it.

3.3 Channel matching strategy

For each contact channel, n8n needs a way to match the click log (session token captured on Shopify) to the Zendesk ticket (created when the customer messages). The matching strategy differs by channel:

Channel	First contact matching	Repeat contact matching	Confidence	CAPI EMQ estimate
WhatsApp	Time window (30 min) between /wa-go click and ticket creation. Phone number stored from Zendesk user API for future contacts.	Phone number → immediate match	High	7.5-8.5
Messenger	Time window (30 min) between /msgr-go click and ticket creation. Page Scoped User ID (PSID) stored from Zendesk identities API.	PSID → immediate match	Medium-High	6.5-7.5

Viber (Oapps)	Time window (30 min) between /viber-go click and ticket creation. Viber user ID stored from Oapps ticket data.	Viber user ID → immediate match	Medium (flag as lower confidence)	5.5-7.0
Email	Session token extracted via regex from ticket body. Email address as fallback identifier.	Email address → immediate match	High when token present	7.0-8.0

```
WhatsApp phone number access: When WhatsApp creates a Zendesk ticket, the
customer's phone number
is stored on the Zendesk user object. Access it via:
  GET /api/v2/users/{requester_id}   → returns user object with 'phone' field

Messenger PSID access: Stored as a messaging identity on the Zendesk user. Access
via:
  GET /api/v2/users/{requester_id}/identities   → find identity with
type='messaging'

Viber user ID: Oapps stores the Viber username, name, and user ID in the ticket
data.
Check ticket description or custom fields that Oapps populates for the exact field
name.
```

3.4 Credentials required for Part 4

Credential	Where to get it	Multi-brand?	Used in
Hetzner Cloud account	hetzner.com — create account	One account, one shared server	Sprint 1
Domain name for n8n	Existing domain or purchase new one. Subdomain recommended (e.g. n8n.yourdomain.com)	One domain, shared server	Sprint 1
Zendesk subdomain + admin API token	Admin Center → Apps and integrations → APIs → Zendesk API → Add API token	Shared — one token	Sprint 4
Zoho Books Client ID + Client Secret *	accounts.zoho.com → API Console → Add Client → Server-based Application	One per brand organization	Sprint 5

Zoho Books Organization ID *	Zoho Books → Settings → Organization Profile → Organization ID	One per brand	Sprint 5
Meta Pixel ID *	Meta Events Manager → Data Sources → [Pixel]	One per brand	Sprint 6
Meta CAPI Dataset ID *	Meta Events Manager → Connect Data Sources → Offline → Create → Dataset ID shown	One per brand	Sprint 6
Meta CAPI Access Token *	Events Manager → Data Sources → [Dataset] → Settings → Conversions API → Generate Access Token	One per brand dataset	Sprint 6
Shopify Warehouse theme — confirmed OS 2.0	Shopify admin → Online Store → Themes — confirm version	One per brand store	Sprint 2

```
IMPORTANT — Zoho Books OAuth2 URLs for Philippines accounts (US datacenter):
  Authorization URL:  https://accounts.zoho.com/oauth/v2/auth
  Access Token URL:   https://accounts.zoho.com/oauth/v2/token
  API Base URL:       https://www.zohoapis.com/books/v3

Scopes required: ZohoBooks.salesorders.READ, ZohoBooks.estimates.CREATE,
ZohoBooks.contacts.READ, ZohoBooks.customfields.READ
```

3.5 Pre-Sprint: Upgrade Shopify stores to OS 2.0

All brand Shopify stores must be running Online Store 2.0 before any tracking code is written. The redirect page setup (Section 3.6) requires OS 2.0 JSON templates. Building on OS 1.0 and migrating later creates rework.

1. In Shopify admin: Online Store → Themes. Check the current theme version.
2. If using Warehouse theme: check for the OS 2.0 version in the theme store and upgrade if not already on OS 2.0.
3. For all brand stores: confirm each is running an OS 2.0 compatible theme before any development begins.
4. After upgrading, verify the theme editor shows the JSON template editor (not the older Liquid template editor). The dropdown at the top of the Customize screen should offer Pages → Create Template.
5. Communicate the upgrade plan to any store managers before executing — theme upgrades can affect store appearance.

3.6 Sprint 1: Server setup (Hetzner + n8n)

Estimated time: 3-4 hours. Sets up the n8n automation server that powers all Part 4 workflows. This server is shared across all brands.

3.6.1 Create Hetzner VPS

1. Go to console.hetzner.com. Log in or create an account.
2. Click Create Server.
3. Select Location: choose the nearest region (Singapore or Tokyo are closest to Philippines).
4. Select Image: Shared vCPU → x86 → CX22 (2 vCPU, 4GB RAM, €4/month). This is sufficient for 10+ brands.
5. Select OS: Ubuntu 24.04.
6. Under Networking: confirm Public IPv4 is enabled.
7. Under SSH keys: add your SSH public key (required — password login will be disabled).
8. Under Firewalls: create a firewall that allows incoming TCP on ports 22 (SSH), 80 (HTTP), 443 (HTTPS). Block all other incoming ports.
9. Click Create & Buy Now.
10. After server creation: note the public IPv4 address. SSH into the server: ssh root@[server-ip]

3.6.2 Configure DNS

1. In your domain registrar's DNS settings, create an A record:
2. Name: n8n (or your chosen subdomain)
3. Type: A
4. Value: [Hetzner server IPv4 address]
5. TTL: 300
6. Wait for DNS propagation (5-30 minutes). Verify: ping n8n.yourdomain.com should resolve to the server IP.

3.6.3 Install Docker and Docker Compose

```
# Run these commands on the Hetzner server via SSH:

apt-get update
apt-get install -y ca-certificates curl gnupg
install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
chmod a+r /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture)
```

```
signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo
$VERSION_CODENAME) stable" \
  | tee /etc/apt/sources.list.d/docker.list > /dev/null
apt-get update
apt-get install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin

# Verify Docker is running:
docker --version
docker compose version
```

3.6.4 Install n8n with Docker Compose and Traefik SSL

n8n's official recommendation uses Traefik as the SSL proxy, not Nginx + Certbot. Follow the official Docker Compose setup exactly.

```
# Create directory and navigate to it:
mkdir n8n-compose && cd n8n-compose

# Create the .env file:
cat > .env << 'EOF'
DOMAIN_NAME=n8n.yourdomain.com
SUBDOMAIN=n8n
GENERIC_TIMEZONE=Asia/Manila
SSL_EMAIL=your-email@domain.com
DATA_FOLDER=/root/n8n-compose
EOF

# Create the compose.yaml file (from official n8n docs):
# Download from: https://docs.n8n.io/deploy/host-n8n/install-options/use-a-cloud-
provider/use-docker-compose
# The official compose.yaml includes: n8n service + Traefik service + PostgreSQL
database
# Do not use a custom compose.yaml — use the official one exactly as documented

# Start n8n:
sudo docker compose up -d

# Verify it's running:
sudo docker compose ps
# Should show: n8n, traefik, and postgres all as 'running'
```

```
# Access n8n:
# Open https://n8n.yourdomain.com in a browser
# Complete the initial setup (create owner account)
```

```
Sprint 1 is complete when ALL of the following are true:
 - Server is accessible via SSH
 - DNS A record resolves to server IP
 - Docker and Docker Compose are installed
 - sudo docker compose ps shows n8n, traefik, and postgres all running
 - https://n8n.yourdomain.com loads the n8n login screen with a valid SSL
certificate
 - n8n owner account created and accessible
```

3.7 Sprint 2: Shopify tracking setup

Estimated time: 1-2 hours per brand store. Sets up the session token capture script and contact button redirect pages on each brand's Shopify store.

```
Prerequisite: Store must be on OS 2.0 before starting this sprint. Confirm before
proceeding.
```

3.7.1 Session token and fbclid capture script

This script runs on every page load. It generates a unique session token and captures the fbclid and UTM parameters from the URL into localStorage. The customer sees nothing.

1. In Shopify admin: Online Store → Themes → [active theme] → ⋯ → Edit code.
2. In the Layout folder, click theme.liquid.
3. Find the </head> closing tag. Paste the script immediately before </head>.
4. Click Save.

```
<script>
(function() {
  // Only run outside the theme editor to avoid editor redirect issues
```

```
  if (window.Shopify && window.Shopify.designMode) return;

  function getParam(key) {
    return new URLSearchParams(window.location.search).get(key);
  }
  function store(key, value) {
    if (value) {
      try { localStorage.setItem(key, value); } catch(e) {}
    }
  }

  // Generate session token if not already set
  if (!localStorage.getItem('hc_session_token')) {
    var token = 'HC-' + Date.now() + '-' + Math.floor(Math.random() * 99999);
    store('hc_session_token', token);
  }

  // Capture fbclid and UTMs from URL (only when present — preserve existing
values)
  var fbclid = getParam('fbclid');
  if (fbclid) store('hc_fbclid', fbclid);

['utm_source','utm_medium','utm_campaign','utm_content','utm_term'].forEach(functi
on(k) {
    var v = getParam(k);
    if (v) store('hc_' + k, v);
  });
})();
</script>

Note: All localStorage keys are prefixed with 'hc_' to avoid conflicts with other
apps.
Update the n8n webhook payloads and Zendesk field mappings to use these key names.
```

3.7.2 Create redirect pages (/wa-go, /viber-go, /msgr-go)

These pages intercept the customer's click on a contact button, log the session token and fbclid to n8n, then redirect the customer to WhatsApp, Viber, or Messenger. The customer experiences a seamless redirect with no visible delay.

Step A — Create a blank custom page template in Shopify:

1. In Shopify admin: Online Store → Themes → [active theme] → Customize.

2. Click the dropdown at the top of the editor (it shows 'Home page' by default).
3. Select Pages → Create Template.
4. Name the template: redirect-blank. Keep Template type as JSON. Click Create Template.
5. In the template editor: in the left sidebar, find the default Page Content section. Click the eye icon to hide it (this creates a truly blank page).
6. Click Save.

Step B — Add redirect script to the template via Edit Code:

1. In Shopify admin: Online Store → Themes → [active theme] → ⋯ → Edit code.
2. In the Templates folder, you will see a new file: page.redirect-blank.json (created in Step A).
3. In the Sections folder: click Add a new section. Name it: redirect-handler. File type: liquid.
4. Paste the redirect handler code below into redirect-handler.liquid. This single section handles all three redirect types based on a data attribute.
5. Click Save.
6. Open page.redirect-blank.json. Add the redirect-handler section to the sections list. Click Save.

```
{% comment %} redirect-handler.liquid {% endcomment %}
<div id='hc-redirect'
     data-target='{{ section.settings.redirect_target }}'
     data-channel='{{ section.settings.channel_name }}'
     data-webhook='{{ section.settings.webhook_url }}'>
</div>

<script>
(function() {
  if (window.Shopify && window.Shopify.designMode) return;

  var el = document.getElementById('hc-redirect');
  var target = el.getAttribute('data-target');
  var channel = el.getAttribute('data-channel');
  var webhook = el.getAttribute('data-webhook');

  function get(k) { try { return localStorage.getItem(k) || ''; } catch(e)
{ return ''; } }

  var payload = {
    session_token: get('hc_session_token'),
    fbclid: get('hc_fbclid'),
    utm_source: get('hc_utm_source'),
```

```
    utm_medium: get('hc_utm_medium'),
    utm_campaign: get('hc_utm_campaign'),
    channel: channel,
    brand: '{{ shop.name }}',
    timestamp: new Date().toISOString()
  };

  // Fire and forget — do not await, do not block redirect
  try {
    fetch(webhook, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(payload),
      keepalive: true
    });
  } catch(e) {}

  // Redirect immediately — webhook is non-blocking
  window.location.replace(target);
})();
</script>

{% schema %}
{
  "name": "Redirect Handler",
  "settings": [
    { "type": "text", "id": "redirect_target", "label": "Redirect URL" },
    { "type": "text", "id": "channel_name", "label": "Channel Name
(WhatsApp/Viber/Messenger)" },
    { "type": "text", "id": "webhook_url", "label": "n8n Webhook URL" }
  ]
}
{% endschema %}
```

Step C — Create the three redirect pages:

1. In Shopify admin: Online Store → Pages → Add page.
2. Title: WhatsApp Redirect (this is internal — customers never see it)
3. Under Template, select: redirect-blank
4. Under URL and handle: set the URL handle to wa-go
5. Click Save.

6. Now go to Online Store → Themes → Customize → open this page via Pages dropdown.

7. Click on the Redirect Handler section. Configure:

8. Redirect URL: https://wa.me/[brand WhatsApp number] (e.g. https://wa.me/639XXXXXXXXX)

9. Channel Name: WhatsApp

10. n8n Webhook URL: https://n8n.yourdomain.com/webhook/[brand]-whatsapp-click

11. Click Save.

12. Repeat for /viber-go (Channel: Viber, Redirect: viber://pa?chatURI=[Viber bot URI]) and /msgr-go (Channel: Messenger, Redirect: https://m.me/[Facebook page username]).

Step D — Update contact buttons on storefront:

```
Replace direct WhatsApp / Viber / Messenger links throughout the theme with the
redirect page URLs.

Old: href='https://wa.me/639XXXXXXXXX'
New: href='/pages/wa-go'

Old: href='viber://pa?chatURI=...'
New: href='/pages/viber-go'

Old: href='https://m.me/PageUsername'
New: href='/pages/msgr-go'

Also update email mailto links to include the session token in the pre-populated
body:
<a href='mailto:sales@brand.com?subject=Inquiry&body=Ref: HC-TOKEN'>Email Us</a>

Use JavaScript in theme.liquid to dynamically inject the token into all mailto
links:
document.querySelectorAll('a[href^="mailto:"]').forEach(function(a) {
  var token = localStorage.getItem('hc_session_token') || '';
  if (token && !a.href.includes('Ref:')) {
    a.href += (a.href.includes('body=') ? '%0A' : '?body=') + 'Ref:%20' + token;
  }
});
```

```
Sprint 2 is complete when ALL of the following are true:
 - theme.liquid script is saved and firing on all pages (verify in browser
localStorage)
```

```
 - /pages/wa-go, /pages/viber-go, /pages/msgr-go all exist and redirect correctly
 - Each redirect page fires the n8n webhook URL before redirecting (verify in n8n
execution log)
 - Payload contains session_token, fbclid (when present), channel, brand,
timestamp
 - Redirect happens in under 500ms — customer sees no delay
 - Contact buttons throughout the storefront use the redirect page URLs
 - Mailto links dynamically include the session token reference in the email body
```

3.8 Sprint 3: Zendesk setup

Estimated time: 2-3 hours. Creates the custom tracking fields in Zendesk and configures RFQ detection. Brand tagging was already set up in Part 3 Sprint 1 — do not duplicate that work, only add the tracking fields below.

3.8.1 Create tracking custom fields in Zendesk

Path: Admin Center → Objects and rules → Tickets → Fields → Add field.

Field name	Field type	Field key	Note
Session Token	Text	hc_session_token	The primary tracking identifier. Never overwrite if already populated.
FB Click ID	Text	hc_fbclid	The Meta fbclid from the ad click. Critical for Meta CAPI attribution.
UTM Source	Text	hc_utm_source	
UTM Medium	Text	hc_utm_medium	
UTM Campaign	Text	hc_utm_campaign	
UTM Content	Text	hc_utm_content	
Contact Channel	Dropdown	hc_contact_channel	Values: WhatsApp, Messenger, Viber, Email. Do NOT name this field 'channel' — reserved system name.
Product Name	Text	hc_product_name	For OOS inquiries — which product the customer asked about
Product SKU	Text	hc_product_sku	
Attribution Confidence	Dropdown	hc_attribution_confidenc	Values: High, Medium, Low. Set by n8n

		e	based on matching method used.

3.8.2 RFQ detection triggers

These Zendesk triggers automatically detect when a customer is requesting a quotation, change the ticket status, and add a tag that n8n monitors. Without this, agents would need to manually trigger the quotation workflow.

1. In Admin Center → Objects and rules → Triggers → Add Trigger.
2. Create the following trigger:

```
Trigger name: RFQ Detection — Keywords

Conditions (Meet ANY):
  Ticket > Comment text > Contains > quote
  Ticket > Comment text > Contains > quotation
  Ticket > Comment text > Contains > rfq
  Ticket > Comment text > Contains > estimate
  Ticket > Comment text > Contains > pricing
  Ticket > Comment text > Contains > how much
  Ticket > Comment text > Contains > magkano
  Ticket > Subject > Contains > quote
  Ticket > Subject > Contains > quotation

Actions:
  Ticket > Add tags > rfq_detected
  Ticket > Status > Open

Note: Do not set status to a custom 'Quotation Needed' status here.
The rfq_detected tag is what n8n monitors — simpler and more reliable.
```

```
Sprint 3 is complete when ALL of the following are true:
 - All 10 custom tracking fields exist in Zendesk and are visible on ticket forms
 - None of the field keys conflict with reserved Zendesk field names
 - RFQ detection trigger is active and firing correctly (test: send a WhatsApp
message
   containing the word 'quote' — confirm rfq_detected tag appears on the ticket)
 - Brand field from Part 3 is still working correctly — confirm no conflicts
```

3.9 Sprint 4: n8n workflow builds

Estimated time: 3-5 days. Builds all automation workflows. All workflows run on the shared n8n server and use brand routing logic to handle multiple brands from a single workflow set.

```
n8n credentials to set up before building workflows:

1. Zendesk — API Token credential:
   In n8n: Settings → Credentials → Add → Zendesk API
   Fields: Subdomain (your-subdomain), Email (admin email), API Token (from
Zendesk Admin Center →
   Apps and integrations → APIs → Zendesk API → API token → Add API token)

2. Zoho Books — OAuth2 credential:
   In n8n: Settings → Credentials → Add → OAuth2 API (Generic)
   Grant Type: Authorization Code
   Authorization URL: https://accounts.zoho.com/oauth/v2/auth
   Access Token URL: https://accounts.zoho.com/oauth/v2/token
   Client ID + Secret: from Zoho API Console → your Books app
   Scope: ZohoBooks.salesorders.READ ZohoBooks.estimates.CREATE
ZohoBooks.contacts.READ

3. HTTP Request (Meta CAPI) — no credential needed; token passed in query string
per request
```

3.9.1 Workflow 1 — Contact click logger (WhatsApp / Viber / Messenger)

Trigger: Webhook (POST from Shopify redirect page). One webhook URL per channel per brand, or one unified webhook with brand routing via the 'brand' field in the payload.

```
WORKFLOW 1 — LOGIC:

Trigger: Webhook node (POST)
  Path: /webhook/contact-click
  Method: POST
  Auth: none (payload is non-sensitive; session tokens have no PII)

Step 1: Set node — normalize and validate payload
  Extract: session_token, fbclid, utm_source, utm_medium, utm_campaign, channel,
brand, timestamp
```

```
  Validate: session_token must not be empty. If empty, stop workflow — nothing to
track.

Step 2: Wait for Zendesk ticket (Wait node — 2 minutes)
  The customer may take 1-3 minutes to open WhatsApp/Viber/Messenger and send
first message.
  Pause workflow for 2 minutes before attempting ticket lookup.

Step 3: Search Zendesk for matching ticket (Zendesk node → Search Tickets)
  Query: 'created>1h brand:[brand_name] status:open'
  If channel = WhatsApp: also filter by requester phone number if available
  If channel = Messenger: filter by requester identity type = messaging (PSID)
  If channel = Viber: use time-window match (tickets created within last 30
minutes for this brand)

Step 4: If ticket found — Update ticket with tracking fields
  IMPORTANT: Use HTTP Request node (not Zendesk node) to ADD tags without
replacing existing ones
  Endpoint: PUT /api/v2/tickets/{ticket_id}
  Body: { 'ticket': { 'custom_fields': [
    { 'id': [hc_session_token field ID], 'value': session_token },
    { 'id': [hc_fbclid field ID], 'value': fbclid },
    { 'id': [hc_utm_source field ID], 'value': utm_source },
    { 'id': [hc_utm_campaign field ID], 'value': utm_campaign },
    { 'id': [hc_contact_channel field ID], 'value': channel },
    { 'id': [hc_attribution_confidence field ID], 'value': confidence_level }
  ]}}

Step 5: If ticket NOT found — Store in n8n data store for later matching
  Key: 'pending_click_' + session_token
  Value: full payload JSON
  TTL: 24 hours (covers delayed messaging behavior)

  n8n will retry matching when new tickets are created (Workflow 2 handles this).

Step 6: Store customer identifier for repeat matching
  WhatsApp: store phone_number → session_token mapping
  Messenger: store psid → session_token mapping
  Viber: store viber_user_id → session_token mapping
  Key: '[channel]_[identifier]'  Value: session_token
  TTL: 365 days (customers may contact again months later)
```

3.9.2 Workflow 2 — New ticket enrichment

Trigger: Zendesk Trigger node (new ticket created event) — OR — Schedule Trigger every 5 minutes checking for tickets without tracking fields.

```
WORKFLOW 2 — LOGIC:

Trigger: Schedule Trigger (every 5 minutes) OR Zendesk webhook on ticket create

Step 1: Get recent Zendesk tickets missing session token
  Query: created_at >= now-10min AND hc_session_token is empty

Step 2: For each untagged ticket — check if identifier matches stored mapping
  Get requester phone number via: GET /api/v2/users/{requester_id}
  Check n8n data store for key: 'whatsapp_' + phone_number
  If found → retrieve session_token and enrich ticket

Step 3: For each untagged ticket — check pending click store
  For Viber/Messenger: check 'pending_click_*' entries where brand matches and
  timestamp is within 30 minutes of ticket created_at
  If found → retrieve session_token and enrich ticket, remove from pending store

Step 4: For email tickets — parse session token from ticket body
  Get first comment text from ticket
  Regex: /Ref:\s*(HC-[0-9]+-[0-9]+)/i
  If match found → use extracted token to enrich ticket
  Tag ticket: token_extracted_email
  If no match → tag: token_missing_email
  (Email address stored for fallback matching on future contacts)
```

3.9.3 Workflow 3 — RFQ detection → Zoho Books estimate creation

Trigger: Zendesk webhook when ticket is tagged with rfq_detected. Creates a pre-populated estimate shell in Zoho Books so agents don't have to manually enter tracking data.

```
WORKFLOW 3 — LOGIC:

Trigger: Zendesk webhook → filter: tag added = rfq_detected

Step 1: Get full ticket data including custom fields
```

```
  GET /api/v2/tickets/{ticket_id}
  Extract: session_token, fbclid, utm fields, brand, channel, requester name

Step 2: Check if estimate already exists for this session token
  GET https://www.zohoapis.com/books/v3/estimates?
organization_id={org_id}&custom_field_value={session_token}
  If estimate exists → skip creation (prevent duplicate estimates)

Step 3: Get or create Zoho Books contact for this customer
  Search by email or phone. If not found, create new contact.

Step 4: Create estimate shell in Zoho Books
  POST https://www.zohoapis.com/books/v3/estimates?organization_id={org_id}
  Body: {
    'customer_id': [contact id from step 3],
    'status': 'draft',
    'custom_fields': [
      { 'label': 'Session Token', 'value': session_token },
      { 'label': 'FB Click ID', 'value': fbclid },
      { 'label': 'UTM Source', 'value': utm_source },
      { 'label': 'UTM Campaign', 'value': utm_campaign },
      { 'label': 'Contact Channel', 'value': channel },
      { 'label': 'Zendesk Ticket ID', 'value': ticket_id },
      { 'label': 'Brand', 'value': brand }
    ],
    'notes': 'Auto-created from Zendesk ticket #' + ticket_id + '. Agent: add
products and send.',
    'line_items': []
  }

Step 5: Update Zendesk ticket with Zoho Books estimate URL
  Add internal note to ticket with estimate URL for agent reference

Step 6: Send email notification to sales team
  To: [brand sales email]
  Subject: New RFQ — [Brand] — [Customer Name] — [Channel]
  Body: Customer name, channel, Zendesk ticket link, Zoho Books estimate link,
session token
```

3.9.4 Workflow 4 — Sales Order created → Meta CAPI

Trigger: Zoho Books webhook (configured in Sprint 5). This is the most critical workflow — it sends the actual conversion event to Meta.

```
WORKFLOW 4 — LOGIC:

Trigger: Webhook node → receives POST from Zoho Books when Sales Order is created

Step 1: Extract Sales Order data from webhook payload
  salesorder_id, customer_id, total, date, custom_fields (session_token, fbclid,
etc.)

Step 2: Validate required fields
  If session_token is empty → log error, stop (cannot attribute without token)
  If fbclid is empty → continue but note lower attribution confidence
  If total = 0 → stop (test order or error)

Step 3: Get customer contact details from Zoho Books
  GET https://www.zohoapis.com/books/v3/contacts/{customer_id}?
organization_id={org_id}
  Extract: email, phone

Step 4: Normalize PII for hashing
  Email: trim whitespace, convert to lowercase
  Phone: remove spaces, dashes, parentheses. Add country code if missing (+63 for
Philippines)

Step 5: Hash PII using n8n Crypto node
  Add Crypto node. Configure:
    Action: Hash
    Type: SHA256
    Value: [normalized email]
    Property Name: hashed_email
    Encoding: hex
  Add second Crypto node for phone:
    Same configuration, Value: [normalized phone], Property Name: hashed_phone

Step 6: Send event to Meta CAPI
  HTTP Request node:
  Method: POST
  URL: https://graph.facebook.com/v19.0/{DATASET_ID}/events?
access_token={CAPI_ACCESS_TOKEN}
  Body (JSON):
  {
    'data': [{
```

```
      'event_name': 'Purchase',
      'event_time': Math.floor(Date.now() / 1000),
      'action_source': 'system_generated',
      'event_id': 'SO-' + salesorder_id,
      'user_data': {
        'em': [hashed_email],
        'ph': [hashed_phone],
        'fbc': 'fb.1.' + Date.now() + '.' + fbclid
      },
      'custom_data': {
        'value': total,
        'currency': 'PHP'
      }
    }],
    'test_event_code': '[TEST_CODE_DURING_TESTING_ONLY — REMOVE IN PRODUCTION]'
  }

Step 7: Log result to Google Sheets (audit trail)
  Append row: timestamp, brand, salesorder_id, session_token, fbclid, total,
capi_response_code
```

3.9.5 Workflow 5 — Daily audit and error sweep

Runs every morning. Checks for tickets still missing tracking data after 24 hours and logs a summary.

```
WORKFLOW 5 — LOGIC:

Trigger: Schedule Trigger — daily at 07:00 PHT

Step 1: Count Zendesk tickets created yesterday with hc_session_token empty
Step 2: Count Zendesk tickets with rfq_detected tag but no corresponding Zoho
Books estimate
Step 3: Check n8n workflow execution log for failures in last 24 hours
Step 4: Send summary email to monitoring person:
  - Tickets missing tracking data: [count] ([percentage] of yesterday's tickets)
  - RFQ tickets without estimate: [count]
  - Workflow failures: [count] (with error details)
  - Sales Orders processed for CAPI: [count]

This audit ensures tracking gaps are surfaced daily rather than discovered weeks
```

```
later.
```

```
Sprint 4 is complete when ALL of the following are true:
 - Workflow 1: /pages/wa-go click → n8n webhook fires → Zendesk ticket gets
tracking fields
 - Workflow 2: New ticket without token → n8n finds pending click log → ticket
enriched
 - Workflow 2: Email ticket → session token extracted from body via regex → field
populated
 - Workflow 3: rfq_detected tag on ticket → Zoho Books estimate created within 60
seconds
 - Workflow 3: Email notification sent to sales team with estimate link
 - Workflow 4: Sales Order created in Zoho Books → Meta CAPI event sent → Test
Events tool shows success
 - Workflow 5: Daily audit email received with correct counts
 - No workflow failures in n8n execution log after 24-hour test run
```

3.10 Sprint 5: Zoho Books setup

Estimated time: 1 hour per brand. Creates the custom tracking fields in Zoho Books and configures the Sales Order webhook.

3.10.1 Create custom fields in Zoho Books

Path: Settings (gear icon, top right) → Customization → Custom Fields → select module → + New Custom Field.

```
CRITICAL: You must create the same custom fields in BOTH the Estimates module AND
the Sales Orders
module. Custom fields created in Estimates do NOT automatically transfer to Sales
Orders when a
quotation is converted. The field names must match exactly for values to inherit
on conversion.


Create the following fields in BOTH modules:
```

Field label	Data type	Module	Note
Session Token	Text (255 chars)	Estimates + Sales Orders	Primary tracking key. Must match exactly in both.

FB Click ID	Text (255 chars)	Estimates + Sales Orders	fbclid value from Meta ad click
UTM Source	Text (100 chars)	Estimates + Sales Orders	
UTM Medium	Text (100 chars)	Estimates + Sales Orders	
UTM Campaign	Text (255 chars)	Estimates + Sales Orders	
Contact Channel	Text (50 chars)	Estimates + Sales Orders	WhatsApp / Messenger / Viber / Email
Zendesk Ticket ID	Text (50 chars)	Estimates + Sales Orders	For cross-system reference
Brand	Text (100 chars)	Estimates + Sales Orders	Brand name — must match Zendesk brand values

3.10.2 Create Sales Order webhook

Step A — Create the webhook action:

1. In Zoho Books: Settings (gear icon) → Automation → Workflow Actions → Webhooks.
2. Click + New Webhook.
3. Name: n8n — Sales Order Created
4. Module: Sales Orders
5. URL & Parameters: POST → enter your n8n webhook URL (https://n8n.yourdomain.com/webhook/zoho-sales-order)
6. Body: select Default Payload (sends all Sales Order fields including custom fields)
7. Check 'I want to secure this webhook'. Enter a Secret Token (12-50 alphanumeric characters). Save this token — you will configure n8n to validate it.
8. Click Save.

Step B — Create the workflow rule:

1. Settings → Automation → Workflow Rules → + New Workflow Rule.
2. Name: Trigger CAPI on Sales Order Creation
3. Module: Sales Orders
4. Workflow Type: Event Based
5. Action Type: Created
6. Click Next. Skip criteria (trigger on all sales orders). Click Next.
7. Under Immediate Actions → + Add Action → Type: Webhook → select the webhook created in Step A.
8. Click Associate → Save.

```
Webhook limits: Maximum 500 webhooks triggered per day. Maximum 1 webhook per
workflow rule.
For high-volume brands, this limit is unlikely to be reached — it would require
500+ sales orders
per day, which is far beyond normal business volume.


Webhook security: Zoho Books sends an X-Zoho-Webhook-Signature header with each
request.
n8n should validate this signature using HMAC-SHA256 with the secret token before
processing.
See n8n documentation for webhook signature validation implementation.
```

```
Sprint 5 is complete when ALL of the following are true:
 - All 8 custom fields exist in BOTH Estimates AND Sales Orders modules with
matching names
 - Zoho Books webhook is configured and workflow rule is active
 - Test: create a Sales Order manually → n8n webhook receives the payload within
30 seconds
 - n8n receives payload containing the custom fields (session_token, fbclid, etc.)
 - Webhook signature validation is working (n8n rejects requests without valid
signature)
```

3.11 Sprint 6: Meta CAPI configuration and testing

Estimated time: 2 hours per brand. Configures the Meta CAPI Dataset and verifies the end-to-end conversion event flow.

```
CRITICAL: The Meta Offline Conversions API was permanently discontinued on May 14,
2025.
Do NOT use the Offline Events Manager or the /offline_conversions endpoint.
All offline conversion tracking now uses the standard Conversions API (CAPI) with
Datasets.
action_source must be 'system_generated' for CRM/offline events.
```

3.11.1 Create Meta CAPI Dataset

1. Go to Meta Events Manager (business.facebook.com → Events Manager).
2. Click + Connect Data Sources.
3. Select Offline.

4. Click Create.
5. Name the Dataset: [Brand Name] Offline Conversions.
6. Link to the brand's ad account.
7. Click Continue. The Dataset ID is shown — copy and store this ID.
8. After creation: click on the Dataset → Settings tab.
9. Under Conversions API section, click Generate Access Token.
10. Copy the access token — store securely in n8n credentials or environment variable.
11. Under Attribution settings: enable Allow attribution window up to 90 days. This is critical for businesses with long sales cycles.

3.11.2 Test Meta CAPI events

1. In Meta Events Manager → the Dataset → Test Events tab.
2. Copy the Test Event Code shown on this page.
3. In n8n Workflow 4, temporarily add the test_event_code field to the CAPI payload: 'test_event_code': '[code from step 1]'
4. Trigger a test Sales Order in Zoho Books with a known session token and fbclid.
5. In the Test Events tab, click Detect events. Within 2-3 minutes, the Purchase event should appear.
6. Verify: event shows with correct value (currency PHP), correct event name (Purchase), and a match quality score (Event Match Quality).
7. Event Match Quality (EMQ) should be 6.0 or higher. If it is below 5.0, check that hashed email and phone are correctly formatted before hashing.
8. Once confirmed working, remove the test_event_code line from the workflow for production use.

```
Expected Event Match Quality by channel (when all fields are correctly populated):
  WhatsApp (phone + fbclid + email): 7.5-8.5  — Strong
  Messenger (PSID + fbclid + email):  6.5-7.5  — Good
  Viber (fbclid + email):             5.5-7.0  — Acceptable
  Email (email + fbclid):             7.0-8.0  — Good when token present

fbc format: 'fb.1.' + [timestamp_when_click_happened] + '.' + [fbclid_value]
This format is required for Meta to attribute the event to the original ad click.
If fbclid is missing, omit the fbc field entirely — do not send an empty or
malformed fbc.
```

```
Sprint 6 is complete when ALL of the following are true:
 - CAPI Dataset created and linked to brand ad account
 - Access token generated and stored in n8n
 - 90-day attribution window enabled in Events Manager
```

```
- Test event appears in Test Events tab with EMQ >= 6.0
- Test event code removed from production workflow
- Payload contains: event_name=Purchase, action_source=system_generated,
  value=[correct amount], currency=PHP, hashed email, hashed phone, fbc
```

3.12 Sprint 7: Multi-brand replication

Repeat the following for every additional brand. Each brand should take 3-5 hours once the primary brand template is established.

Step	Task	Notes
1	Upgrade brand Shopify store to OS 2.0 if not already done	Prerequisite for Sprint 2 equivalent
2	Add session token script to brand theme.liquid	Identical script to primary brand
3	Create /wa-go, /viber-go, /msgr-go redirect pages on brand store	Update redirect target URLs and n8n webhook URLs for this brand
4	Update contact buttons on brand storefront	Use this brand's redirect page URLs
5	Add brand to Zendesk Brand dropdown field	Admin Center → Tickets → Fields → Brand → add new value
6	Create brand-specific Zendesk auto-tagging triggers	One trigger per channel for this brand
7	Create custom tracking fields in this brand's Zoho Books organization	Same 8 fields in both Estimates and Sales Orders
8	Create Sales Order webhook in this brand's Zoho Books	New webhook pointing to same n8n URL with brand identifier in payload
9	Create Meta CAPI Dataset for this brand's ad account	New Dataset ID and Access Token
10	Update n8n workflows to handle this brand's routing	Add brand to routing logic in Workflows 3 and 4
11	Test full funnel for this brand	Click ad → contact → RFQ → Sales Order → CAPI event in Test Events
12	Add brand workspace to Zoho Analytics Master Dashboard	After Part 3 Sprint 4 onboarding for this brand

3.13 QA and full funnel test cases

Run all test cases below after completing all sprints for each brand. Do not sign off until all test cases pass.

Test case	Steps	Expected result
Meta → WhatsApp → Quotation → CAPI	1. Click a Meta ad. 2. Check localStorage for hc_session_token and hc_fbclid. 3. Click WhatsApp button (/wa-go). 4. Send message in WhatsApp. 5. Check Zendesk ticket for tracking fields. 6. Send message containing 'quote'. 7. Check Zoho Books for estimate. 8. Convert estimate to Sales Order. 9. Check Meta Test Events.	localStorage has token + fbclid. n8n webhook fired. Zendesk ticket has all tracking fields. Zoho Books estimate created within 60 seconds with tracking fields. CAPI Purchase event appears in Test Events with EMQ >= 6.0.
Meta → Messenger → Quotation	1. Click Meta ad. 2. Click Messenger button (/msgr-go). 3. Send message in Messenger. 4. Check Zendesk ticket.	Ticket enriched with session token. Channel field = Messenger. Attribution confidence set appropriately.
Meta → Viber → Quotation	1. Click Meta ad. 2. Click Viber button (/viber-go). 3. Send message in Viber. 4. Check Zendesk ticket.	Ticket enriched. Channel field = Viber. Attribution confidence = Medium (lower confidence flagged).
Meta → Email	1. Click Meta ad. 2. Click email link. 3. Check email body for Ref: HC-token. 4. Send email. 5. Check Zendesk ticket.	Email body contains session token reference. After ticket creation, n8n extracts token via regex. Ticket has hc_session_token populated.
Repeat customer — WhatsApp	1. Customer contacts via WhatsApp for a second time (days later). 2. New Zendesk ticket created. 3. Check if previous session token or new token is used.	n8n finds stored phone → session_token mapping. New ticket enriched with the most recent session token (from their most recent ad click).
No fbclid — organic visitor	1. Visit store directly (no ad click). 2. Click WhatsApp button. 3. Check n8n webhook payload. 4. Check Zendesk ticket.	hc_fbclid is empty in payload. Ticket still gets hc_session_token and channel fields. Workflow continues without fbclid. CAPI event NOT sent (no ad click to attribute to).
Sales Order with missing tracking fields	1. Create a Sales Order manually without going through the tracked funnel. 2. Check n8n workflow execution.	n8n Workflow 4 receives webhook. Detects session_token is empty. Logs the gap in audit log. Does NOT send CAPI event. Audit workflow flags this Sales Order in daily report.

```
Part 4 final acceptance — complete when ALL of the following are true:
 - All 7 test cases pass for primary brand
 - All 7 test cases pass for each additional brand
```

```
  - Daily audit email shows tracking gap rate below 10%
  - CAPI EMQ score is 6.0 or higher for at least WhatsApp and Email channels
  - No n8n workflow failures after 72-hour production test run
  - Zoho Books Sales Order webhook firing within 30 seconds of SO creation
  - Zoho Books estimate created within 60 seconds of rfq_detected Zendesk tag
  - All custom fields present in both Estimates and Sales Orders in all brand
organizations
  - All brand Shopify stores have redirect pages working and logging correctly
```

3.14 Maintenance notes — Part 4

Task	Frequency	Details
Meta CAPI access token refresh	When token expires or on rotation schedule	In Meta Events Manager → Dataset → Settings → Conversions API → Generate new token. Update in n8n workflow credentials.
Review daily audit email	Every morning	Monitoring person checks tracking gap rate and workflow failures. Investigate if gap rate exceeds 10%.
Review CAPI Event Match Quality	Monthly	In Meta Events Manager → Dataset → Overview → Event Match Quality. If score drops below 5.0, check hashing normalization logic in Workflow 4.
Add new brand to system	When new brand is launched	Follow Sprint 7 multi-brand replication steps. Add brand to Zendesk dropdown, create brand triggers, create Zoho Books custom fields, create CAPI Dataset.
Shopify theme updates	When theme is updated on any brand store	Verify redirect pages (/wa-go etc.) still exist and function after any theme update. Verify theme.liquid tracking script is still present.
n8n version updates	Quarterly	Run: sudo docker compose pull && sudo docker compose up -d. Test all 5 workflows after update.
Server health check	Monthly	SSH into Hetzner server. Check disk space: df -h. Check n8n containers: sudo docker compose ps. Check logs: sudo docker compose logs --tail=100 n8n.
Zoho Books OAuth token	Zoho may require re-authorization	If n8n Zoho Books HTTP requests start failing with 401, re-authenticate the OAuth2 credential in n8n Settings → Credentials.

4. System-wide Maintenance & Operations

4.1 How Part 3 and Part 4 work together

Part 3 and Part 4 are independent systems that run in parallel. Here is how they interact in daily operations:

Scenario	Part 3 role	Part 4 role
Creative fatigue begins	Alert fires — CTR drop detected in Meta Ad Insights. Monitoring person notified.	Tracks which customers from fatigued ads still convert — helps distinguish fatigue from audience exhaustion.
Inquiry volume drops	Alert fires — inquiry rate decline detected in Zendesk/GA4 data.	Identifies if drop is ad-attributed or organic — shows whether declining inquiries came from tracked ad clicks.
New Sales Order created	Zoho Books connector will pick this up in next daily sync — updates pipeline metrics.	Immediately triggers CAPI event to Meta. Attribution happens in real time, not next-day.
New brand launched	Add brand workspace to Zoho Analytics. Connect 5 data sources. Configure alerts.	Add brand to Zendesk, Shopify, Zoho Books, n8n workflows, Meta CAPI Dataset.

4.2 Daily operations — monitoring person

Time	Task	Tool	Action if issue found
08:00	Read daily digest email from Zoho Analytics	Email	Review each brand's key metrics. Flag any that are trending down.
08:15	Read daily audit email from n8n	Email	If tracking gap rate > 10%, escalate to developer. If workflow failures, investigate.
08:30	Check Master Dashboard in Zoho Analytics	Zoho Analytics	Review cross-brand scorecard. Any brands in red — escalate to brand team.
As alerts arrive	Triage incoming email alerts	Email	WATCH: log and monitor. WARNING: notify brand team. CRITICAL: notify brand team + business owner immediately.
End of week	Check brand_untagged count in Zendesk	Zendesk	If count is rising, brand tagging triggers may need adjustment. Report to developer.

4.3 Alert response SOPs

Standard operating procedures for each alert type:

Creative Fatigue — WARNING or CRITICAL

1. Open the Meta Ads dashboard for the affected brand.
2. Find the specific ad mentioned in the alert.
3. Check the ad's performance over the last 14 days — confirm the CTR trend is real, not a data anomaly.
4. If confirmed: prepare a replacement creative from the creative library.
5. WARNING: Schedule the replacement creative for the next rotation cycle (within 3 days).
6. CRITICAL: Pause the affected ad immediately. Launch replacement creative same day.
7. Monitor new creative CTR for the first 3 days. Report outcome to monitoring person.

Audience Saturation — WARNING or CRITICAL

1. Identify the affected adset.
2. Check the audience size and how long the adset has been running.
3. WARNING: Either expand the audience (add new interest layers or increase lookalike percentage) OR rotate the creative to refresh the experience for the same audience.
4. CRITICAL: Pause the adset immediately. The audience is burned. Create a new adset with an expanded or different audience targeting the same product.
5. Do not simply lower the budget on a saturated adset — this extends the saturation period without solving it.

Delivery Problem — CRITICAL

1. This is the most urgent alert — budget is being spent but ads are not being shown.
2. Immediately check Meta Business Manager → Account Quality for any policy flags or payment issues.
3. Check if payment method on the ad account is valid and not expired.
4. Check if any campaigns have been flagged or restricted.
5. If no policy issues found, check if the campaign budget has been exhausted.
6. Report findings to business owner within 1 hour.

4.4 Final system acceptance checklist

Run this checklist after all sprints are complete for all brands. Do not sign off until every item is confirmed.

#	Check	How to verify
1	GA4 installed on all brand stores	GA4 Realtime shows live visitors from each brand store
2	UTMs on all Meta ads for all brands	GA4 sessions table shows facebook/paid_social per brand
3	Meta Ads connector syncing — all brands	Each brand Zoho Analytics workspace — Ad Insights table has data from last 24 hours
4	Shopify connector syncing — all brands	Each workspace — Orders table has recent records
5	GA4 connector syncing — all brands	Each workspace — Sessions table shows yesterday's data
6	Zendesk connector syncing	Tickets table shows recent records with Brand field populated
7	Zendesk brand tagging — all brands	brand_untagged count near zero. All recent tickets have Brand field set.
8	Zoho Books connector syncing — all brands	Each workspace — Estimates and Sales Orders tables have recent records
9	Inquiry Rate Query Table working per brand	Returns non-null inquiry_rate_pct values with brand column
10	Query Tables A, B, C built — all brands	FR Stores exclusion filter confirmed in all three tables for all brand workspaces
11	All 4 dashboards built per brand	Open each brand workspace — all widgets load with real data
12	All 11 alerts active per brand — brand name in subject	Settings → Alerts — all 11 show Active. Test alert received with correct brand name.
13	Master Dashboard built	Master workspace shows all brands in cross-brand views
14	Portfolio alerts configured	4 portfolio-level alerts active in Master workspace
15	n8n server running	sudo docker compose ps shows all containers running. https://n8n.yourdomain.com accessible.
16	Shopify tracking script on all brand stores	Check browser localStorage for hc_session_token after visiting each brand store
17	Redirect pages working — all brands	Visit /pages/wa-go on each brand store — redirect fires, n8n webhook receives payload

18	Zendesk tracking fields created — all brands	Admin Center → Tickets → Fields — all 10 custom fields present
19	Zendesk RFQ trigger active	Send 'quote' via WhatsApp test — rfq_detected tag appears on ticket
20	n8n Workflow 1 — click logging working	n8n execution log shows successful runs after clicking redirect pages
21	n8n Workflow 2 — ticket enrichment working	Zendesk tickets created after redirect page click have tracking fields populated
22	n8n Workflow 3 — RFQ → Estimate working	rfq_detected ticket → Zoho Books estimate created within 60 seconds
23	Zoho Books custom fields — all brands, both modules	Custom fields present in Estimates AND Sales Orders in all brand organizations
24	Zoho Books Sales Order webhook active	Create test SO → n8n execution log shows webhook received within 30 seconds
25	Meta CAPI Datasets created — all brands	Each brand has Dataset ID and Access Token. 90-day attribution window enabled.
26	CAPI events tested — all brands	Test Events tool shows Purchase event with EMQ >= 6.0 for each brand
27	Daily audit email working	Audit email received each morning with correct counts
28	All 7 QA test cases passed per brand	See Section 3.13 — all test cases documented and signed off
29	Credentials stored securely	All tokens and secrets stored in n8n credentials or password manager — not in workflow code
30	Monitoring person briefed	Monitoring person understands daily operations (section 4.2), alert routing, and escalation procedures

4.5 Adding a new brand — cross-system checklist

When a new brand is added to the portfolio, complete the following across both systems:

Part 3 (Analytics & Monitoring) — new brand

1. Upgrade new brand Shopify store to OS 2.0
2. Create GA4 property for new brand store. Install GA4 on store. Add UTMs to all Meta ads.
3. Create new Zoho Analytics workspace: [New Brand] — Ads Monitoring

4. Connect all 5 data sources for new brand
5. Add new brand to Zendesk Brand dropdown field
6. Create auto-tagging triggers for new brand's contact channels
7. Build all 4 dashboard views
8. Build Query Tables A, B, C
9. Configure all 11 alerts with new brand name in subject lines
10. Configure daily digest email
11. Add new brand workspace to Master Dashboard

Part 4 (Conversion Tracking) — new brand

1. Add session token tracking script to new brand's Shopify theme.liquid
2. Create /wa-go, /viber-go, /msgr-go redirect pages on new brand store
3. Update contact buttons on new brand storefront
4. Create Zoho Books custom fields in new brand organization (Estimates + Sales Orders)
5. Create Sales Order webhook in new brand's Zoho Books
6. Create Meta CAPI Dataset for new brand's ad account
7. Add new brand routing logic to n8n workflows
8. Run all 7 QA test cases for new brand
9. Run 72-hour production test — confirm daily audit email shows new brand

4.6 Credential management and rotation schedule

Credential	Rotation requirement	Action required
Meta CAPI Access Token	Does not expire automatically, but rotate every 6 months for security	Events Manager → Dataset → Settings → Generate new token → update in n8n
Meta Ads Connector (Zoho Analytics)	Re-authenticate every 60 days	Zoho Analytics → Data Sources → Meta Ads → Re-authenticate before expiry
Shopify Connector (Zoho Analytics)	Does not expire unless revoked	Re-authenticate only if sync fails with auth error
GA4 Connector (Zoho Analytics)	Re-authenticate if Google account password changes	Zoho Analytics → Data Sources → GA4 → Re-authenticate

Zendesk API Token	Rotate annually or if compromised	Zendesk Admin Center → APIs → API token → Create new → update in n8n → delete old
Zoho Books OAuth2 (n8n)	Zoho may expire refresh tokens after inactivity	If n8n calls fail with 401 → n8n Settings → Credentials → Zoho Books → Re-authenticate
Hetzner SSH key	Rotate if server admin changes	Add new SSH key in Hetzner console → verify login → remove old key
n8n Encryption Key	Never rotate without full backup	Rotating the encryption key breaks all existing credentials — requires re-entering all credentials